

```

import cmath

def cardano_formula(a, b, c, d):
    if a == 0:
        raise ValueError("Dies ist keine kubische Gleichung, da a = 0 ist.")

    # Normalisieren der Koeffizienten
    p = c / a - (b ** 2) / (3 * a ** 2)
    q = (2 * b ** 3) / (27 * a ** 3) - (b * c) / (3 * a ** 2) + d / a

    # Diskriminante berechnen
    delta = (q / 2) ** 2 + (p / 3) ** 3

    # Lösungen berechnen
    if delta > 0:
        u = (-q / 2 + cmath.sqrt(delta)) ** (1 / 3)
        v = (-q / 2 - cmath.sqrt(delta)) ** (1 / 3)
        roots = [u + v - b / (3 * a)]
    elif delta == 0:
        u = (-q / 2) ** (1 / 3)
        roots = [2 * u - b / (3 * a), -u - b / (3 * a)]
    else:
        u = cmath.exp(cmath.log(-q / 2 + cmath.sqrt(delta)) / 3)
        v = cmath.exp(cmath.log(-q / 2 - cmath.sqrt(delta)) / 3)
        roots = [u + v - b / (3 * a), -(u + v) / 2 - b / (3 * a) + cmath.sqrt(3)
        * (u - v) * 1j / 2, -(u + v) / 2 - b / (3 * a) - cmath.sqrt(3) * (u - v) * 1j /
        2]

    return roots

# Beispielaufruf für die Gleichung x^3 + 3x^2 + 3x + 1 = 0
a, b, c, d = 1, 3, 3, 1
lösungen = cardano_formula(a, b, c, d)
print("Lösungen der kubischen Gleichung:", lösungen)

```

```

import cmath

def cardano_formula(a, b, c, d):
    if a == 0:
        raise ValueError("Dies ist keine kubische Gleichung, da a = 0 ist.")

    # Normalisieren der Koeffizienten
    p = c / a - (b ** 2) / (3 * a ** 2)
    q = (2 * b ** 3) / (27 * a ** 3) - (b * c) / (3 * a ** 2) + d / a

    # Diskriminante berechnen
    delta = (q / 2) ** 2 + (p / 3) ** 3

```

```

# Lösungen berechnen
if delta > 0:
    u = (-q / 2 + cmath.sqrt(delta)) ** (1 / 3)
    v = (-q / 2 - cmath.sqrt(delta)) ** (1 / 3)
    roots = [u + v - b / (3 * a)]
elif delta == 0:
    u = (-q / 2) ** (1 / 3)
    roots = [2 * u - b / (3 * a) , -u - b / (3 * a)]
else:
    u = cmath.exp(cmath.log(-q / 2 + cmath.sqrt(delta)) / 3)
    v = cmath.exp(cmath.log(-q / 2 - cmath.sqrt(delta)) / 3)
    roots = [u + v - b / (3 * a) , -(u + v) / 2 - b / (3 * a) + cmath.sqrt(3)
* (u - v) * 1j / 2 , -(u + v) / 2 - b / (3 * a) - cmath.sqrt(3) * (u - v) * 1j /
2]

return roots

# Benutzer-Eingabe
print("Geben Sie die Koeffizienten der kubischen Gleichung  $ax^3 + bx^2 + cx + d = 0$  ein:")
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))
d = float(input("d: "))

try:
    lösungen = cardano_formula(a, b, c, d)
    print("Lösungen der kubischen Gleichung:", lösungen)
except ValueError as e:
    print(e)

```

Cardanische Formel in Python

Dieses Programm berechnet die Lösungen einer kubischen Gleichung der Form $ax^3 + bx^2 + cx + d = 0$ mithilfe der cardanischen Formel. Es berücksichtigt reale und komplexe Wurzeln.

Code

```

import cmath

def cardano_formula(a, b, c, d):
    if a == 0:
        raise ValueError("Dies ist keine kubische Gleichung, da a = 0 ist.")

    # Normalisieren der Koeffizienten
    p = c / a - (b ** 2) / (3 * a ** 2)
    q = (2 * b ** 3) / (27 * a ** 3) - (b * c) / (3 * a ** 2) + d / a

```

```

# Diskriminante berechnen
delta = (q / 2) ** 2 + (p / 3) ** 3

# Lösungen berechnen
if delta > 0:
    u = (-q / 2 + cmath.sqrt(delta)) ** (1 / 3)
    v = (-q / 2 - cmath.sqrt(delta)) ** (1 / 3)
    roots = [u + v - b / (3 * a)]
elif delta == 0:
    u = (-q / 2) ** (1 / 3)
    roots = [2 * u - b / (3 * a)] [-u - b / (3 * a)]
else:
    u = cmath.exp(cmath.log(-q / 2 + cmath.sqrt(delta)) / 3)
    v = cmath.exp(cmath.log(-q / 2 - cmath.sqrt(delta)) / 3)
    roots = [u + v - b / (3 * a)] [-(u + v) / 2 - b / (3 * a) + cmath.sqrt(3)
* (u - v) * 1j / 2] [-(u + v) / 2 - b / (3 * a) - cmath.sqrt(3) * (u - v) * 1j /
2]

    return roots

# Benutzer-Eingabe
print("Geben Sie die Koeffizienten der kubischen Gleichung  $ax^3 + bx^2 + cx + d = 0$  ein:")
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))
d = float(input("d: "))

try:
    lösungen = cardano_formula(a, b, c, d)
    print("Lösungen der kubischen Gleichung:", lösungen)
except ValueError as e:
    print(e)

```

Erklärung

1. **Eingabe der Koeffizienten:** Der Benutzer gibt die Werte für a, b, c und d ein.
2. **Prüfung auf kubische Gleichung:** Es wird sichergestellt, dass a ungleich 0 ist.
3. **Normalisierung:** Die Gleichung wird auf eine reduzierte Form gebracht.

4. Diskriminante: Die Diskriminante wird berechnet, um die Art der Wurzeln zu bestimmen.
5. Berechnung: Die Lösungen werden je nach Diskriminante berechnet.
6. Ausgabe: Die Lösungen werden ausgegeben, oder eine Fehlermeldung, falls die Eingabe ungültig ist.

```
import math

def cardano_method():
    print("Dieses Programm löst die kubische Gleichung der Form  $x^3 + ax^2 + bx + c = 0$ :")

    # Eingabe der Koeffizienten
    a = float(input("Bitte geben Sie a ein: "))
    b = float(input("Bitte geben Sie b ein: "))
    c = float(input("Bitte geben Sie c ein: "))

    # Transformation der Gleichung in die reduzierte Form  $x^3 + px + q = 0$ 
    p = b - (a ** 2) / 3
    q = (2 * a ** 3) / 27 - (a * b) / 3 + c

    # Berechnung der Diskriminante
    discriminant = (q / 2) ** 2 + (p / 3) ** 3

    if discriminant > 0:
        # Eine reelle Wurzel
        u = math.cbrt(-q / 2 + math.sqrt(discriminant))
        v = math.cbrt(-q / 2 - math.sqrt(discriminant))
        root = u + v - a / 3
        print(f"Eine reelle Lösung ist: {root:.7f}")
    elif discriminant == 0:
        # Alle Wurzeln reell, mindestens zwei gleich
        u = math.cbrt(-q / 2)
        root1 = 2 * u - a / 3
        root2 = -u - a / 3
        print(f"Zwei gleiche reelle Lösungen sind: {root1:.7f}")
        print(f"Eine weitere reelle Lösung ist: {root2:.7f}")
    else:
        # Drei verschiedene reelle Wurzeln
        r = math.sqrt(-(p / 3) ** 3)
        theta = math.acos(-q / (2 * r))
```

```

root1 = 2 * math.cbrt(r) * math.cos(theta / 3) - a / 3
root2 = 2 * math.cbrt(r) * math.cos((theta + 2 * math.pi) / 3) - a / 3
root3 = 2 * math.cbrt(r) * math.cos((theta + 4 * math.pi) / 3) - a / 3
print(f"Drei verschiedene reelle Lösungen sind: {root1:.7f}, {root2:.7f},
{root3:.7f}")

# Programm ausführen
cardano_method()

```

```

import cmath

def cardano_method():
    print("Dieses Programm löst die kubische Gleichung der Form  $x^3 + ax^2 + bx + c = 0$ :")

    # Eingabe der Koeffizienten
    a = float(input("Bitte geben Sie a ein: "))
    b = float(input("Bitte geben Sie b ein: "))
    c = float(input("Bitte geben Sie c ein: "))

    # Transformation der Gleichung in die reduzierte Form  $x^3 + px + q = 0$ 
    p = b - (a ** 2) / 3
    q = (2 * a ** 3) / 27 - (a * b) / 3 + c

    # Berechnung der Diskriminante
    discriminant = (q / 2) ** 2 + (p / 3) ** 3

    # Berechnung der Wurzeln mit cmath
    u = cmath.exp(cmath.log(-q / 2 + cmath.sqrt(discriminant)) / 3)
    v = cmath.exp(cmath.log(-q / 2 - cmath.sqrt(discriminant)) / 3)

    # Berechnung der Lösungen
    root1 = u + v - a / 3
    root2 = -(u + v) / 2 - a / 3 + cmath.sqrt(3) * (u - v) * 1j / 2
    root3 = -(u + v) / 2 - a / 3 - cmath.sqrt(3) * (u - v) * 1j / 2

    # Ausgabe der Wurzeln
    print(f"Die Lösungen sind: {root1}, {root2}, {root3}")

# Programm ausführen
cardano_method()

```
